

CORSO DI ALGORITMI E STRUTTURE DATI

Prof. ROBERTO PIETRANTUONO

PROBLEMA 1

Si consideri la cronologia di una serie di eventi storici rappresentata da un array di interi, ciascuno dei quali indica l'anno di occorrenza di un evento.

Si scriva un programma che, dato in ingresso detto array, ed un numero intero positivo Y , calcoli il maggior numero di eventi occorsi in un periodo di Y anni, nonché l'anno del primo evento e dell'ultimo evento in quel periodo. Si noti che, dato un anno x , il periodo di Y anni che inizia nell'anno x è l'intervallo di tempo che intercorre dal primo giorno dell'anno x all'ultimo giorno dell'anno $x+Y-1$. Se esiste più di un periodo Y con lo stesso numero di eventi, il programma deve riportare quello più antico.

Sample Input

$Y = 4$

Eventi storici: [1, 3, 3, 5, 8, 10, 11, 11, 15, 16, 18, 19, 20, 25]

Sample Output

Numero di eventi massimo in un periodo di $Y = 4$ anni è: 4, occorsi tra l'anno 8 e l'anno 11

PROBLEMA 2

Si supponga di disporre di una scacchiera $N \times N$. Si scriva un algoritmo di **backtracking** per determinare il numero di modi in cui è possibile posizionare N regine in modo tale che nessuna coppia di regine si possa attaccare a vicenda. Quindi, una soluzione richiede che due regine non condividano la stessa riga, colonna o diagonale (si supponga $N > 3$). L'input è il valore N . In output, il programma deve riportare il **numero** di modi in cui è possibile posizionare N regine in modo tale che nessuna coppia di regine si possa attaccare a vicenda.

Sample Input

$N = 4$

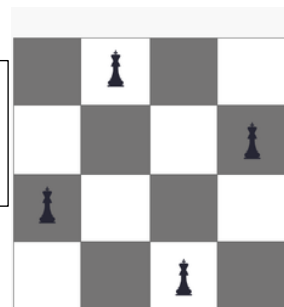
$N = 5$

Sample Output

Numero di configurazioni = 2

Numero di configurazioni = 10

Esempio di
configurazione
corretta per $N = 4$



PROBLEMA 3

Un artigiano deve svolgere N lavori (ordini dai clienti). Egli può svolgere un solo lavoro al giorno. Per ogni i -esimo lavoro, è noto T_i ($1 \leq T_i \leq 1000$), un intero che indica il tempo in giorni impiegato dall'artigiano per finire il lavoro. Per ogni giorno di ritardo accumulato prima di iniziare a lavorare l' i -esimo ordine, l'artigiano deve pagare una penale di S_i ($1 \leq S_i \leq 10000$) centesimi di euro. Si implementi un programma per trovare la sequenza dei lavori che comporta la minima somma di penali.

INPUT

L'input inizia con un singolo intero positivo su una riga a sé stante, indicante il numero dei casi di test successivi. Le righe successive descrivono il caso di test; la prima di tali righe contiene un numero intero N ($1 \leq N \leq 1000$), le successive N righe contengono ciascuna due numeri: il tempo impiegato e la penale di ogni attività (la prima coppia di numeri si riferisce al lavoro $i=1$, seconda coppia il lavoro $i=2$, ecc.).

OUTPUT

Per ogni caso di test, il programma dovrà stampare la sequenza dei lavori che comporta una penale minima. Ogni lavoro è identificato dall'ordine in cui è apparso in input, i . Tutti i numeri interi devono essere posizionati su una sola riga di output e separati da uno spazio. Se sono possibili più soluzioni, stampare la prima soluzione in ordine lessicografico.

Osservazione: data una coppia di lavori i, j , la decisione di schedulare prima i o prima j non impatta sul costo totale dei restanti $N-2$ lavori; non è dunque necessario controllare ogni possibile permutazione.

Sample Input

```
2
4
3 4
1 1000
2 2
5 5
3
123 123
3 3
5 5
```

Sample Output

```
2 1 3 4
1 2 3
```